Overleaf + Claude: papers, research code and data, with AI

Start here

This guide sets up AI-assisted collaboration around a paper in three levels. Each level builds on the one before without changing it, so every co-author uses only what they need.

Level	What	Lives on	Template
1. Manuscript	The paper's LaTeX, notes and marked edits	Overleaf	`overleaf-paper-template.zip`
2. Code	Julia package with the models, simulation jobs, figure scripts	GitHub	`research-project-template.zip`
3. Data	Simulation results	The group's cluster, synced to laptops	(part of the Level 2 template)

Find your role:

Role	Works with	Read
Writer	The manuscript only: in the Overleaf browser editor, or in VS Code with Claude	Level 1
Writer and researcher	Manuscript, code and data in one VS Code window	Levels 1, 2 and 3
Researcher	Code and data (laptop and cluster), no manuscript	Levels 2 and 3

Within Level 1, writers either stay in the browser (nothing to install) or become *Claude writers*, who open the same manuscript in VS Code through Git. The project owner (usually the corresponding author) sets each level up once per paper.

Level 1: Manuscript

How it works

The paper lives on Overleaf, as usual. Co-authors who want Claude's help also open it in VS Code through Overleaf's Git integration, and every change Claude makes is marked in colour so anyone can accept or reject it.

Who	Works in	Syncing with Overleaf
Browser writers	Overleaf, exactly as today	Automatic
Claude writers	VS Code + Claude Code + the GitDoc extension	Automatic on every save
Claude writers who use Git themselves	VS Code + Claude Code, without GitDoc	Pull and push by hand

What you need. The project owner needs an Overleaf premium plan; then every member of the project can use Git with it. Claude writers need a Claude subscription for Claude Code. Browser writers need nothing new.

How everyone talks to Claude. Each co-author has a note command, such as `\noteAlice{...}`, and the PDF numbers the notes: **ALICE#1:**, **ALICE#2:**, and so on. A Claude writer says "address Alice#1", and Claude does what the note asks, marks every change, and removes the note.

The template (`overleaf-paper-template.zip`) keeps the visible top folder minimal:

```
main.tex     all the text, plus one \newreviewer line per person
biblio.bib   references
figures/     images
tools/       reviewmarkup.sty (the review commands), README.md (legend for
             browser writers), githooks/ (Git safety check)
.claude/     CLAUDE.md (Claude's rules), CLAUDE.local.md.example,
settings.json
.vscode/     shared VS Code settings, recommended extensions, one-time setup
task
.github/     instructions for Copilot (points to .claude/CLAUDE.md)
```

Setting up a paper (project owner)

You set up one Overleaf **template project** once, through Git, and copy it for each new paper.

Once: the template project. Use Git rather than uploading the zip in the browser, so the hidden folders (`.claude`, `.vscode`, `.github`) arrive intact.

1. In Overleaf, create a Blank Project called "Paper template". Open the **Integrations** side panel → **Git**, copy the Git URL, and press **Generate token**. The username is always `git`; the token is the password.
2. On your computer:

```
git clone https://git.overleaf.com/PROJECT-ID paper-template
cd paper-template
git rm -q -r .                                  # remove Overleaf's sample files
unzip -q ../overleaf-paper-template.zip -d /tmp/tpl
cp -r /tmp/tpl/overleaf-paper-template/. .
git add -A && git commit -m "Start from template" && git push
```

3. Back in Overleaf, check that the project compiles and that the hidden folders appear in the file tree.

For each paper.

1. In Overleaf's project list, **Copy** the template project and type the paper's name in the Copy dialog. The template stays untouched, and the copy gets its own Git address. Writers always clone the copy, never the template.
2. In `main.tex`, replace the placeholder title, authors, emails and affiliations with the paper's own, and add one line per co-author next to Claude's, with a colour name or a model and values:

```
\newreviewer{Claude}{DarkRed}
\newreviewer{Alice}{DarkOrange}
\newreviewer{Bob}{RGB}{0,110,60}
```

Each line creates that person's commands (note, margin note, edit), e.g. `\noteBob` and `\editBob`. Colour names come from xcolor's svgnames list (Navy, DarkRed, ForestGreen, …); mixes like `blue!60!black` and `{HTML}{FF8800}` work too.

3. Fill in the paper conventions at the end of `.claude/CLAUDE.md` (language variant, tense, notation).

4. **Share** the project with your co-authors as usual.

If you change the preamble, keep `xcolor` and any page-layout package such as `geometry` before `\usepackage[draft]{tools/reviewmarkup}`.

Browser writers: working with Claude from Overleaf

Nothing changes in how you write. You gain one habit: notes in the text, written with your own command.

```
This paragraph explains the tidal model in far too much detail.
\noteAlice{Shorten this paragraph to three sentences.}
```

The PDF shows it inline in your colour as **ALICE#1: Shorten this paragraph…**. For a short remark beside the text, use `\marginAlice{...}` instead. Both kinds share one numbering per person, counted from the top of the paper. In two-column journal layouts the draft PDF is made wider so margin notes fit, while the final version keeps the journal's page size.

Getting Claude to act on a note. Ask a Claude writer to "address Alice#1" (or several: "Alice#1 and Alice#3"). Claude quotes the note back, does the work with marked edits, and removes the note; if it has a question, the note is replaced by a Claude note. Notes nobody asks about are left alone. Removing a note renumbers your later ones, so use the numbers from the latest PDF.

Reading marked edits. In draft mode:

You see	Meaning
New text in colour, old text struck out next to it	A short edit, waiting to be accepted
`[deleted: …]` in pale colour, then `[…]` in full colour	A longer edit spanning paragraphs, equations or headings
A box starting with **CLAUDE#1:**	A note or question by Claude

The colour tells you who made the change: Claude is dark red. You can mark your own edits the same way, with `\editAlice[old]{new}`.

Accepting or rejecting. Keep only the new text (accept) or only the old text (reject), deleting the `\editClaude[...]{...}` wrapper, or ask a Claude writer to "accept Claude's edits in the introduction".

Use notes, not Overleaf comments or Track Changes. Claude cannot see Overleaf's comments or tracked changes, and Overleaf warns that changes arriving through Git can displace them.

Final version. In `main.tex`, change `\usepackage[draft]{tools/reviewmarkup}` to `[final]`: notes disappear, every remaining marked edit shows as accepted, and the page returns to its normal size.

Claude writers: installation and joining a paper

Install once. Windows works natively; no WSL is needed.

	macOS	Windows	Linux
VS Code	code.visualstudio.com	code.visualstudio.com	code.visualstudio.com
Git	Type `git --version` in Terminal and accept the install offer	Git for Windows (git-scm.com), default options	Usually preinstalled; otherwise your package manager
Claude Code	VS Code extension "Claude Code", then sign in	same	same
GitDoc	VS Code extension "GitDoc" (skip it if you use Git yourself)	same	same
TeX (optional)	MacTeX	MiKTeX or TeX Live	TeX Live from your package manager

TeX and the LaTeX Workshop extension give you a local PDF that recompiles on every save. Without TeX, don't install LaTeX Workshop (or disable it for the paper folder), and read the PDF on Overleaf.

Join a paper (once per paper).

1. **Get the address and a token.** Open the paper on Overleaf, then the **Integrations** side panel → **Git**. Copy the Git URL and press **Generate token**. Make one token per computer: it works for all your papers, expires after a year, and if a computer is lost you delete just its token in Overleaf's Account Settings.

2. **Clone.** In VS Code: Ctrl/Cmd+Shift+P → **Git: Clone** → paste the URL → choose a folder. Log in with username `git` and the token as password. VS Code names the folder after Overleaf's project ID and doesn't ask for a name, so rename the folder to the paper's name right away, before opening it. Alternatively clone from a terminal, choosing the name: git clone https://git.overleaf.com/PROJECT-ID "Paper name". (If you rename it later on Linux, run the setup task again so Git finds the stored token.)
3. **Open** the cloned folder and accept three prompts: **trust** the folder, **allow** the automatic task, and **install** the recommended extensions. The task is a one-time setup: it tells Git how to merge your work with Overleaf's, switches on a check that stops unfinished conflicts reaching Overleaf, and on Linux prepares storage for your token.
4. **Enter the token once more at the first sync.** Save any change; a small password box appears at the top of VS Code. Your token is then remembered: by the system keychain on macOS and Windows, and on Linux in a file inside the clone's hidden `.git` folder, readable only by you and off-limits to Claude.
5. **Check GitDoc is watching.** A small mirror icon appears in the status bar. If it doesn't, run **GitDoc: Disable** and then **GitDoc: Enable** from the Command Palette (Enable alone does nothing).

You are set: from now on, saving syncs with Overleaf.

Working with Claude day to day

Open the paper folder in VS Code, open the Claude Code panel, and ask in plain words. Claude follows `.claude/CLAUDE.md`: it pulls the latest text first, marks every change with `\editClaude`, and never invents references.

You say	Claude does
“Address Alice#1”	Quotes the note, does what it asks with marked edits, removes it
“Address all of Alice's notes in the introduction”	Proposes a plan first, then the same for each note
“Tighten the abstract”	Small marked edits in the abstract
“Read the methods and leave notes, don't edit”	Claude notes, no text changes
“Accept Claude's edits in section 2”	Keeps the new text there and removes the markup

You say	Claude does
“Resolve the Git conflict”	Merges both versions sensibly and explains what it chose

Without being asked, Claude also fixes clear spelling mistakes it notices (as marked edits), and fixes errors in cited bibliography entries directly in `biblio.bib`, leaving a short margin note at the first citation. Claude Code shows each change and asks before applying it, unless you allow edits for the session.

Syncing happens on every save. Press Ctrl/Cmd+S, or switch to another window (that saves too). A few seconds later GitDoc commits, pulls Overleaf's latest text, merges it, and pushes; browser writers see your change within seconds. With TeX installed, the same save also recompiles your local PDF. Save before asking Claude to edit, so your unsaved typing and Claude's changes don't collide.

Getting co-authors' changes without having edited anything: click the sync arrows in VS Code's status bar. Overleaf cannot push to you; your side has to ask. Claude always pulls before starting a task.

When you and a browser writer changed the same lines, Git stops with a conflict: `main.tex` shows both versions between `<<<<<<<` and `>>>>>>>` lines. Either ask Claude to “resolve the Git conflict”, or click **Accept Current Change** (yours), **Accept Incoming Change** (Overleaf's) or **Accept Both Changes** above each block, then save. GitDoc finishes the merge and pushes. A safety check refuses any commit that still contains conflict markers, so a half-resolved file never reaches Overleaf. If GitDoc ever offers a **Force Push** button, don't press it: Overleaf refuses it anyway.

Good habits.

- Tell co-authors which section you are working on, and don't run Claude on a paragraph someone is editing in the browser at that moment.
- Address notes a few at a time, and read the result before saving again.
- Keep requests specific: "shorten this to three sentences, keep the second result" works better than "improve this".

If you use Git yourself (no GitDoc): pull before you start, push when you finish, or ask Claude to "pull" and "commit and push". Overleaf has a single branch (`main` in recent projects, `master` in older ones); never create others.

Optional extras

Personal instructions: CLAUDE.local.md

`.claude/CLAUDE.md` holds the rules everyone shares. For your own preferences, copy `.claude/CLAUDE.local.md.example` to `.claude/CLAUDE.local.md` and edit the copy. Copy it, don't rename it: the example is shared, and renaming would delete it for everyone at the next sync. Your file is in `.gitignore`, so it never reaches Overleaf, and Claude reads it at the start of every session.

```
- My name: Alice. "My notes" are notes written with \noteAlice.
- Talk to me in: English. Write paper text in: English.
- How bold to be: conservative. Ask before rewriting more than a sentence.
- Syncing: I use GitDoc, so never commit or push yourself.
```

Your file can change how Claude behaves for you, but not the shared safety rules: Claude still marks every edit, never invents references, and never force-pushes or creates branches.

Linux: keep the token in your keyring

By default the setup task stores the token in a plain-text file inside the clone (readable only by you). To keep it encrypted in GNOME Keyring or KDE Wallet instead, set up Git's libsecret helper once, before cloning:

Linux	Commands
Fedora	`sudo dnf install git-credential-libsecret`, then `git config --global credential.helper libsecret`
Fedora Atomic (e.g. Silverblue)	In a Fedora distrobox: `sudo dnf install git-credential-libsecret`, then `distrobox-export --bin /usr/libexec/git-core/git-credential-libsecret --export-path ~/.local/bin`; on the host: `git config --global credential.helper libsecret`
Arch	`git config --global credential.helper /usr/lib/git-core/git-credential-libsecret` (included with git)

Linux	Commands
Debian, Ubuntu	`sudo apt install libsecret-1-dev build-essential`, `sudo make --directory=/usr/share/doc/git/contrib/credential/libsecret`, then `git config --global credential.helper /usr/share/doc/git/contrib/credential/libsecret/git-credential-libsecret`

Check which helpers are active with `git config --show-origin --get-all credential.helper`. If a paper was already cloned with the plain-text store, remove it there: `git config --unset credential.helper`, delete `.git/overleaf-credentials`, and enter the token once more at the next sync.

Updating the template or an existing paper

In the template's clone (or a paper's clone), pull, copy the changed files from the new template zip over the old ones, then commit and push:

```
git pull
cp /tmp/tpl/overleaf-paper-template/tools/reviewmarkup.sty tools/   # for example
git commit -am "Update template files" && git push
```

For existing papers, keep a copy of the paper's conventions at the end of .claude/CLAUDE.md before overwriting it, and update only the files under `tools/`, `.claude/` and `.vscode/`; never overwrite a paper's `main.tex` or `biblio.bib`.

Security notes

- **A token opens all your Overleaf projects,** not just one paper. Keep one token per computer, never share it, and delete it in Account Settings if a computer or a synced folder is compromised. Claude is barred from reading stored tokens.
- **Shared files run on your computer.** Anyone who can edit the Overleaf project can change `.vscode/tasks.json` (run when you open the folder) and `tools/githooks/pre-commit` (run on every commit), and `.claude/CLAUDE.md` steers what Claude does. Allow the automatic task only for papers whose co-authors you trust, and after pulling you can review changes to these files with `git log -p -- tools .vscode .claude .github`.

- **The plain-text token file on Linux** is readable by any program running as you, and travels with backups or cloud-synced folders. Use the keyring option above if that matters.

Alternative route: Claude Desktop with OverleafMCP (experimental)

Co-authors who would rather chat than use VS Code can connect the Claude Desktop app straight to the Overleaf project with OverleafMCP: no local copy, no Git, no GitDoc. Treat it as an experiment until you have run the test below.

Setup (once per paper). Note the project ID (the part after `/project/` in the Overleaf address) and your Git token, add OverleafMCP to Claude Desktop's configuration as its README shows, and restart Claude Desktop. Then create a Claude **Project** for the paper with these instructions:

```
This project is a LaTeX paper on Overleaf, reached through the overleaf
tools.
At the start of each conversation, read .claude/CLAUDE.md with read_file and
follow it, except its Git and compiling instructions: the overleaf tools
handle
syncing. To change text, re-read the section just before writing, then use
write_section, never write_file. Mark every change with \editClaude.
```

Using it. Ask as in VS Code ("address Alice#1"); each change goes straight to Overleaf as a commit.

Test first. OverleafMCP writes a whole section at a time, so a browser writer's change made between Claude reading and writing that section could be silently undone. Have a co-author edit a sentence while Claude works on the same section, and check that their edit survives. If it doesn't, use this route only for sections nobody else is editing. Claude cannot compile on this route, so check Overleaf's PDF after each batch.

Troubleshooting

When in doubt, ask Claude what happened: it can read the Git state and fix most of these. View → Output → **Git** shows every Git command GitDoc ran, with its errors.

Symptom	Cause	Fix
Saving does nothing; no mirror icon in the status bar	GitDoc started without noticing the repository	Command Palette: **GitDoc: Disable**, then **GitDoc: Enable**

Symptom	Cause	Fix
Saves never reach Overleaf, or every sync asks for the token	Git cannot store the token, so pushes wait for a password box	Run the setup task, then enter `git` and the token at the next sync; check with `git pull` twice (the second must not ask)
Sync Changes shows arrows both ways (e.g. 1↓ 1↑) but nothing merges	Git was not told how to merge diverged copies	Run the setup task (or `git config pull.rebase false`), then click Sync Changes
`<<<<<<<` lines in `main.tex`, or a commit "blocked"	You and a browser writer changed the same lines	Ask Claude to "resolve the Git conflict", or use the Accept buttons, then save
GitDoc warns of conflicting changes and offers Force Push	Overleaf has newer text	Don't force; click the sync arrows or save again
"Authentication failed" or "repository not found"	Token expired (after a year), deleted, or mistyped	Generate a new token in Overleaf; enter it at the next prompt
The setup task never ran	It wasn't allowed when the folder opened	Command Palette → **Tasks: Run Task** → **Paper: one-time setup**
Overleaf won't compile after an edit or note	It starts inside an equation, environment or `{...}` group and ends outside it	Overleaf's log names the line; ask Claude to fix it, or move the brace
"Undefined control sequence \noteBob"	No `\newreviewer` line for that person	Add one in `main.tex`, next to Claude's
Claude addressed the wrong note	Numbers shifted since your PDF was compiled	Check the quote Claude gives before acting; recompile and use the new number
Claude can't see a comment	It is an Overleaf comment, not a note in the text	Paste it to Claude, or rewrite it as a note
Overleaf comments or tracked changes vanished	Changes arriving through Git can displace them	Use notes and marked edits instead

Symptom	Cause	Fix
A margin note sits in the gap between columns	It falls exactly at a column break	Move it a sentence earlier or later, or make it inline
Margin notes cut off at the page edge	A page-layout package such as `geometry` is loaded after `reviewmarkup`	Load it before `tools/reviewmarkup`
LaTeX Workshop reports an error at every save	TeX is not installed	Install TeX, or disable LaTeX Workshop for the folder
Git errors about rate limits	Too many automatic syncs	Keep syncing tied to saves; don't enable timed autosave or VS Code's auto-fetch

Level 2: Code

For researchers, and for writers who are also researchers. A private GitHub repository holds the Julia package with the models, the simulation jobs and the figure scripts. The manuscript is unchanged: writers and researchers can sit inside one VS Code window, or a researcher can skip the manuscript entirely.

How it works

```
PROJECT/                  research repository (GitHub)
├── simulations/          Julia package MyPaperPackage: src/, test/, run/
(jobs), out/ (logs)
├── analysis/             plotting environment: plots.example.jl, output/
├── data/                 simulation results (Level 3; gitignored)
├── manuscript/           the Overleaf clone (gitignored: a separate
repository)
├── syncdata.sh, .conf    data transfer (Level 3)
└── CLAUDE.md             research rules; imports the manuscript's rules
```

Role	Clones
Writer and researcher	The research repository, with the manuscript cloned inside it as `manuscript/`

Role	Clones
Researcher (laptop)	The research repository only
Cluster	The research repository only (Level 3)

The template needs Julia 1.10 (the long-term-support version) or later.

Setting up a project (project owner)

As with Overleaf, keep one template repository on GitHub: push the unzipped `research-project-template.zip` to an empty private repository once, and tick **Template repository** in its Settings. For each project:

1. On GitHub, **Use this template → Create a new repository** (private), and clone it.
2. Rename the package and commit. The script replaces `MyPaperPackage` everywhere, gives the package a fresh UUID (each package needs its own, or Julia mixes them up between projects), and deletes itself:

```
./rename-package.sh TwistedBilayer
git add -A && git commit -m "Rename package"
```

3. Install both environments and commit the resulting `Manifest.toml` files, which record the exact package versions:

```
julia --project=simulations -e 'using Pkg; Pkg.instantiate()'
julia --project=analysis -e 'using Pkg; Pkg.instantiate()'
```

Julia 1.11 and later find the package from `analysis/` by themselves (a `[sources]` entry). On Julia 1.10, link it once before instantiating, and commit `analysis/Manifest.toml` so co-authors don't have to: `cd analysis && julia --project=. -e 'using Pkg; Pkg.develop(path="../simulations")'`.

Joining a project

1. Clone the research repository.
2. Writers and researchers: clone the paper into it as `manuscript` (Overleaf: Integrations → Git; token as in Level 1): `git clone https://git.overleaf.com/PROJECT-ID manuscript`.

3. Open the **research folder** (not `manuscript/`) in VS Code. Trust it, allow its automatic task (it runs the manuscript's one-time setup inside `manuscript/`), and install the recommended extensions (Julia, Claude Code, GitDoc, LaTeX Workshop).
4. Install the two environments as above.

Working with code and figures

- **Syncing.** Saving a manuscript file syncs it with Overleaf exactly as in Level 1. GitDoc is limited to `manuscript/`, so research code is never committed automatically: commit and push it yourself, or ask Claude to "commit and push the analysis changes".
- **Claude** follows the research rules in `CLAUDE.md` everywhere and the manuscript rules inside `manuscript/`. It never touches `data/`, never submits jobs unless asked, and asks for approval before global changes.
- **Julia.** Two environments: `simulations` (the package and its jobs) and `analysis` (plots). The VS Code REPL starts in `analysis`; switch in the status bar. Model code belongs in `simulations/src/`; scripts only call package functions. After changing the package, run its tests: `julia --project=simulations -e 'using Pkg; Pkg.test()'`.
- **Figures.** Copy `analysis/plots.example.jl` to `plots.jl` and adapt it. Each figure is a builder function (data as positional arguments, looks as keywords), saved to `analysis/output/` at journal-column size with LaTeX fonts; `loaddata` warns if the data came from uncommitted code. When a figure is final, copy it into `manuscript/figures/` (by hand or by asking Claude) and save: Overleaf receives it.

Troubleshooting

Symptom	Cause	Fix
Research files get committed automatically	VS Code was opened on `manuscript/` instead of the research folder, or GitDoc's file pattern was changed	Open the research folder; keep `"gitdoc.filePattern": "**/manuscript/**"` in its `.vscode/settings.json`
"Package MyPaperPackage not found"	The environment isn't installed, or on Julia 1.10 the package was never linked	Run the instantiate commands above; on Julia 1.10, link the package as described

Level 3: Data

Simulations run on the group's cluster, and the cluster's storage holds the reference copy of each project's data (around 1 GB). Laptops pull what they need with `syncdata.sh`, which is part of the Level 2 repository.

How it works

This level assumes each researcher already has **passwordless SSH** to the cluster, with an entry like this in `~/.ssh/config`:

```
Host mycluster
    HostName login.cluster.example.es
    User alice
```

The alias (`mycluster`) is the only thing the repository knows about the cluster; user names stay in each person's own SSH configuration. On the cluster, the repository's `data/` is a link to the project's storage, so jobs write straight there; on laptops, `data/` is an ordinary folder filled by `syncdata.sh`. Either way, scripts always read and write `data/`.

Setting up (project owner, once)

Edit `syncdata.conf` and commit it:

```
REMOTE=mycluster                                # the SSH alias everyone
defines
REMOTE_PATH=/storage/mygroup/myproject/data   # the project's data folder on
the cluster
LOCAL_PATH=data
EXCLUDE=("*.tmp" "scratch/")
```

Personal deviations (for example a different local folder) go in a `syncdata.local.conf`, which is gitignored.

On the cluster: running jobs

Once per person, clone the repository and link `data/` to the project's storage:

```
git clone git@github.com:GROUP/PROJECT.git && cd PROJECT
ln -s /storage/mygroup/myproject/data data
```

To run a computation, copy `simulations/run/job.example.jl` under a new name (say `bands.jl`) and adapt it; nothing else needs renaming. Commit the code, then submit from the repository root:

```
sbatch simulations/run/bands.jl
```

The job installs and precompiles anything new once (so after a `git pull` there is nothing else to do), starts one Julia worker per Slurm task, and writes `data/bands.fig1.jld2`, saving each case as soon as it is ready. The file includes `meta`: the Git commit, whether the code had uncommitted changes, Julia version, date, node, worker count and job ID, so every figure can be traced to the code that made it. Slurm logs go to `simulations/out/`; they are only for monitoring and can be deleted.

On a laptop: getting the data

Connect the VPN if needed, then from the repository root:

Command	Does
`./syncdata.sh`	Pulls new and changed files from the cluster into `data/`; never deletes anything
`./syncdata.sh -n`	Dry run: shows what would be copied
`./syncdata.sh --push`	Copies local files to the cluster, after asking (rarely needed)

Claude can run the pull for you, but never pushes. On Windows, run the script in WSL. Recent macOS versions ship a reduced `rsync`; if the script complains, `brew install rsync`.

At publication

Archive the final data on Zenodo and cite its DOI in the paper's data availability statement.

Troubleshooting

Symptom	Cause	Fix
Job stops with “Run from the repository root”	`sbatch` was run from another folder	`cd` to the repository root, then `sbatch simulations/run/yourjob.jl`
Job runs but no log appears	`simulations/out/` is missing, so Slurm silently drops the log	Restore it (kept in Git by a `.gitkeep`): `git checkout simulations/out`
`syncdata.sh` can't reach the cluster	VPN off, or the alias in `syncdata.conf` isn't in your `~/.ssh/config`	Connect the VPN; check that `ssh mycluster` logs in without a password
A figure warns its data came from uncommitted code	The job ran with uncommitted changes in `simulations/`	Commit the code and rerun the job before the figure goes into the paper